Sistema distribuido de reducción adaptativa de ruido con filtrado FIR y espectral sobre microcontroladores

Nicolás Florez Jiménez, Tamara Vanessa Cajiao Molina, Santiago Robles Obando, Santiago Boza Quiros
Escuela de Ingeniería en Computadores, Instituto Tecnológico de Costa Rica, Cartago, Costa Rica
CE 1110 Análisis de Señales Mixtas, II Semestre 2026

***Resumen*—This paper presents a three-node wired system that acquires a noisy signal, characterizes its noise and reduces it in quasi real time with two independent strategies. An RP2040 acquisition node samples the input at 16 kHz with DMA, removes the DC component, applies a Hann window and a 256-point FFT for diagnosis, and streams 256-sample blocks to an ESP32 through SPI frames that carry a header, a sequence identifier and a CRC-8 checksum. The ESP32 runs two processing paths with separate state: a 75-tap Kaiser FIR low-pass filter with linear phase and a block spectral method based on a 1024-point radix-2 FFT, 50 % overlap-add and an adaptive relative threshold that preserves conjugate symmetry. The output is played through an 8-bit DAC followed by a second-order Sallen-Key reconstruction filter, and a host computer is used only for storage, parameter tuning and playback. A Python model with a direct DFT, FIR and IIR candidates and automatic figure generation guided the design; the embedded filters reproduce the model within 0.0001 dB of output SNR. For noise outside the useful band the FIR filter improved the SNR by about 60 dB and the spectral method by about 19 dB, with worst-case block times of 8.8 ms and 18.9 ms against budgets of 16 ms and 32 ms. When noise overlaps the useful band neither method improves the SNR without removing useful energy, which quantifies the trade-off between noise reduction and distortion.**

***Index Terms*—noise reduction, FIR filter, spectral filtering, FFT, embedded systems, distributed acquisition**

I. Introducción

Toda señal adquirida desde un sensor llega acompañada de interferencias: tonos de equipos cercanos, ruido de banda ancha del propio circuito o perturbaciones impulsivas. Reducirlas en el mismo sistema embebido que adquiere la señal, sin depender de una computadora, exige elegir con cuidado la frecuencia de muestreo, el tamaño de bloque, el formato numérico y el método de filtrado, porque memoria, tiempo de cálculo y ancho de banda del enlace son limitados [1].

Este trabajo diseña, implementa y evalúa un sistema de tres nodos cableados: un nodo de adquisición y análisis en una Raspberry Pi Pico y dos rutas de procesamiento independientes en una ESP32, una temporal (FIR) y otra espectral (FFT/IFFT) con cálculo de métricas. Ambos métodos se diseñaron primero en un modelo de Python que también sirve de referencia para validar el firmware.

La Sección II resume los fundamentos teóricos. La Sección III describe la arquitectura, el enlace entre nodos y el diseño de ambos filtros. La Sección IV presenta y analiza los resultados, y las Secciones V y VI cierran con conclusiones y recomendaciones.

II. Marco teórico

Convolución y respuesta en frecuencia. Un sistema lineal e invariante en el tiempo responde con $y[n] = x[n] * h[n]$, que en frecuencia equivale a $Y(e^{j\omega}) = X(e^{j\omega})H(e^{j\omega})$ [2]. La respuesta $H(e^{j\omega}) = |H(e^{j\omega})|e^{j\angle H(e^{j\omega})}$ es compleja: la magnitud fija la atenuación de cada frecuencia y la fase su retardo.

FIR, IIR y transformada Z. La ecuación de diferencias

$$y[n] = \sum_{k=0}^{M} b_k x[n-k] - \sum_{k=1}^{N} a_k y[n-k] \tag{1}$$

tiene función de transferencia $H(z) = \sum b_k z^{-k}/(1 + \sum a_k z^{-k})$. Un sistema causal es estable si todos sus polos están dentro del círculo unitario, con región de convergencia $|z| > \max_k |p_k|$. Un FIR ($a_k = 0$) solo tiene polos en $z = 0$, es siempre estable y, con coeficientes simétricos, tiene fase lineal y retardo constante de $M/2$ muestras [3].

DFT, FFT y ventanas. La DFT de N muestras, $X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}$, cuesta $O(N^2)$; la FFT radix-2 la calcula en $O(N \log_2 N)$ si $N = 2^m$ [4]. La resolución es $\Delta f = f_s/N$. Truncar un bloque produce fuga espectral; la ventana Hann reduce los lóbulos laterales a cambio de un lóbulo principal más ancho [5], y con 50 % de solapamiento sus copias suman una constante, lo que permite reconstruir por superposición y suma [6]. Para que la IFFT de un espectro modificado sea real se debe conservar $X[N-k] = X^*[k]$.

Métricas. Con referencia limpia $s[n]$ y salida alineada $\hat{s}[n]$ sobre L muestras:

$$\mathrm{MSE} = \tfrac{1}{L}\textstyle\sum(s-\hat{s})^2, \quad \mathrm{SNR_{dB}} = 10\log_{10}\dfrac{\sum s^2}{\sum(s-\hat{s})^2}, \tag{2}$$

y $E_x = \sum |x[n]|^2$. La mejora es $\Delta\mathrm{SNR} = \mathrm{SNR}_{out} - \mathrm{SNR}_{in}$.

III. Diseño e implementación

III-A. Arquitectura y flujo de datos

La Fig. 1 (Anexo A) muestra el sistema. El nodo 1 (RP2040) muestrea la entrada por su ADC de 12 bits a $f_s = 16$ kHz con DMA, en bloques de $N = 256$ (16 ms, $\Delta f = 62.5$ Hz) [7]. Elimina la media de cada bloque y, sobre una copia, aplica

Hann y FFT para estimar el nivel, la frecuencia dominante y la saturación. Las muestras sin componente DC se envían a la ESP32, donde dos rutas con estado independiente ejecutan el FIR (nodo 2) y el método espectral (nodo 3), permitido por el enunciado al ser un microcontrolador de doble núcleo [1], [8]. La computadora solo almacena, ajusta parámetros y solicita la reproducción; no filtra. La terminal de la PC funciona como interfaz local del nodo 3: muestra el método activo, la energía conservada, el MSE y la SNR antes y después que calcula la ESP32.

Enlace entre nodos. Cada bloque viaja por SPI a 2 MHz en una trama de 1032 bytes: cabecera (2), identificador de secuencia (4), 256 muestras en float32 (1024), CRC-8 con polinomio $x^8 + x^2 + x + 1$ (1) y un byte de relleno [9]. La ESP32, esclava SPI [10], controla el flujo con una línea READY y rechaza tramas con longitud, cabecera o CRC inválidos o con muestras no finitas; el identificador revela saltos, duplicados y retrocesos. Cualquier pérdida o cola llena aborta la captura y queda registrada. El enlace con la PC usa paquetes PCM16 con índice y CRC-32 a 1 Mbaud; 2 Mbaud falló con el puente USB de la placa.

Justificación conjunta. Las componentes generadas no superan 4 kHz, por lo que $f_s = 16$ kHz deja Nyquist en 8 kHz con margen [2]. El flujo útil es $16\,000 \times 4$ B $= 64$ kB/s, holgado frente a los 250 kB/s del SPI a 2 MHz. Hacia la PC viajan tres señales PCM16 (96 kB/s) sobre unos 99 kB/s útiles a 1 Mbaud: una ocupación cercana al 97 %, verificada con una captura completa sin pérdidas. Los bloques de 256 (FIR) y 1024 con salto de 512 (espectral) son potencias de dos [1]. [PENDIENTE: evidencia de antialias antes del ADC con micrófono real]

III-B. Filtro temporal (nodo 2)

Con banda de paso hasta 1.6 kHz, rechazo desde 2.4 kHz, rizado de 0.1 dB y atenuación mínima de 60 dB se compararon un FIR por ventana Kaiser [11] y tres IIR (Tabla I). Se eligió el FIR de orden 74: es incondicionalmente estable (sus 74 polos están en $z = 0$, ROC $|z| > 0$), tiene fase lineal con retardo constante de 37 muestras (2.3 ms) y obtuvo la mayor SNR. Los IIR requieren menos operaciones, pero su retardo de grupo varía en la banda útil y deforma los transitorios. Su diagrama de polos y ceros y su respuesta en frecuencia están en el Anexo A.

Tabla I
CANDIDATOS DE FILTRO TEMPORAL (SIMULACIÓN, CASO 2)

Filtro	Orden	MAC	Retardo	SNR (dB)
FIR Kaiser	74	75	37	65.1
Butterworth	20	50	18.4–24.7	15.5
Chebyshev I	10	25	13.3–20.6	14.8
Elíptico	6	15	6.0–10.6	18.6

En la ESP32 el FIR procesa muestra a muestra con un anillo de 75 posiciones en float32 y coeficientes exportados desde Python. La salida se convierte a PCM16 con saturación explícita y, para la reproducción, a 8 bits del DAC, lo que limita el rango dinámico a unos 48 dB. [PENDIENTE: FIR medido vs calculado]

III-C. Filtro espectral y evaluación (nodo 3)

El método procesa bloques de $N = 1024$ ($\Delta f = 15.625$ Hz) con salto de 512:

1. multiplicar el bloque por Hann;
2. calcular la FFT radix-2 [4];
3. proteger los bins bajo f_{min} (1.6 kHz);
4. marcar como ruido cada bin no protegido cuya energía $|X[k]|^2$ supere un umbral relativo (0.05) a la máxima energía de esa región, junto con sus vecinos dentro de un ancho total de 100 Hz;
5. multiplicar esos bins y sus conjugados por un factor real (0.1);
6. calcular la IFFT, volver a aplicar Hann y reconstruir por superposición y suma normalizada por $\sum w^2$ [6].

El umbral se recalcula en cada bloque, por lo que la selección sigue la interferencia si cambia de frecuencia o de nivel [12]. Los cuatro parámetros se cambian en ejecución desde la terminal, sin recompilar. El mismo nodo acumula, en doble precisión, MSE, SNR de entrada y salida, energía útil conservada, atenuación de la interferencia y tiempo por bloque.

III-D. Reconstrucción analógica

La salida del DAC pasa por un divisor de 10 kΩ/10 kΩ, un seguidor (LM358, amplificador A) y un Sallen-Key pasabajos de ganancia unitaria (amplificador B) con $R_1 = R_2 = 4.7$ kΩ, $C_1 = 22$ nF de realimentación y $C_2 = 10$ nF a tierra [13]–[15], alimentado a 5 V. Su frecuencia de corte $f_c = 1/(2\pi\sqrt{R_1 R_2 C_1 C_2}) \approx 2283$ Hz y su $Q = \frac{1}{2}\sqrt{C_1/C_2} \approx 0.74$ atenúan las imágenes del DAC antes del amplificador. Se usó 22 nF, valor comercial disponible, en lugar de los 20 nF de un Butterworth exacto ($Q = 0.707$, 2394 Hz); el cambio produce un ligero realce de $+0.3$ % bajo 1 kHz. Con el sistema en marcha se midieron 5 V en la alimentación y 0.815 V en la entrada y la salida del seguidor, la mitad del nivel medio del DAC, como predice el divisor.

IV. RESULTADOS Y ANÁLISIS

Criterios de aceptación, fijados antes de las pruebas finales: con ruido fuera de banda, $\Delta\text{SNR} \geq 10$ dB y energía útil conservada ≥ 90 %; en todos los casos, tiempo máximo por bloque menor que su duración (16 ms FIR, 32 ms espectral); cero pérdidas de bloque. Para el filtro analógico, error de ganancia ≤ 15 % por tono con variabilidad ≤ 10 %. La concordancia entre la salida de la placa y el modelo de Python, $100(1 - \text{RMSE}/\text{RMS})$, debe ser ≥ 90 %: un error de implementación 20 dB por debajo de la señal es menor que el efecto que se mide (mejoras de 18 a 60 dB) y deja margen para la cuantización PCM16 y el uso de float32 en la placa. [PENDIENTE: confirmar con los resultados reales]

Equivalencia modelo–firmware. Los mismos casos se evaluaron en Python (float64) [16], [17] y en la ESP32 (float32) sobre 95 963 muestras, tras compensar las 37 muestras del

FIR. La SNR de salida coincide con diferencias ≤ 0.0001 dB y el MSE en sus tres cifras significativas (Tabla II); más de 150 pruebas automáticas en Python y las pruebas en C del receptor validan el núcleo C y la FFT propia contra NumPy. En la sesión con el sistema completo, el caso 1 procesado por la ruta espectral en vivo dio 29.9764 dB frente a 29.9760 dB del modelo. Una captura física de 6 s con el micrófono entregó 96 000 muestras a 16 000.26 muestras/s, sin bloques perdidos ni recortes, con un tiempo máximo de cálculo de 16.3 ms frente a 32 ms disponibles.

Tabla II
RESULTADOS EN ESP32 (95 963 MUESTRAS, $f_s = 16$ kHz, 160 MHz, 30/10/2026)

Caso	Método	SNR_{in} (dB)	ΔSNR (dB)	MSE	Útil (%)	t_{max} (ms)
1: tono 4 kHz	FIR	10.0	59.05	1.7×10^{-8}	100.0	8.83
	Espectral	10.0	19.97	1.3×10^{-4}	100.0	18.86
2: banda 2.5–3.8 kHz	FIR	5.0	60.14	1.5×10^{-8}	100.0	8.83
	Espectral	5.0	18.35	2.3×10^{-4}	100.0	18.95
3: banda 0.6–1.5 kHz	FIR	0.0	0.003	2.2×10^{-2}	100.0	8.80
	Espectral	0.0	0.002	2.2×10^{-2}	100.0	18.84

Comparación de métodos. Con ruido fuera de banda (casos 1 y 2) el FIR mejora la SNR unos 60 dB porque atenúa al menos 60 dB toda la banda de rechazo, mientras el espectral queda limitado a unos 20 dB por su factor de 0.1. Ambos conservan prácticamente toda la energía útil. Los tiempos máximos (8.8 ms y 18.9 ms) cumplen sus presupuestos con margen a 160 MHz; la versión final corre a 240 MHz con el procesamiento en un núcleo distinto de las interrupciones. El banco de filtros ocupa 53 272 bytes de memoria de trabajo compartida. La latencia algorítmica del FIR es su retardo de 2.3 ms más el bloque de 16 ms; la del espectral es del orden de una ventana completa, 64 ms, más 2.3 ms de retardo añadido para alinearla con el FIR.

Compromiso ruido–distorsión. En el caso 3 el ruido comparte la banda útil y ninguno de los dos métodos mejora la SNR, porque proteger la señal útil también protege el ruido superpuesto. Al bajar f_{min} a 600 Hz, el espectral mejora 5.9 dB con factor 0.1 pero conserva solo 72 % de la energía útil, frente a 1.8 dB y 90 % con factor 0.8 (Fig. 2, Anexo A).

[PENDIENTE: Tabla de P1–P5 en hardware con promedio $\pm$ desviación de 3 repeticiones: tono (P1), tres tonos (P2), audio con ruido de banda ancha (P3), transitorio con ruido impulsivo (P4), superposición (P5); pérdidas de bloque y errores CRC observados]

V. CONCLUSIONES

El sistema cumple la arquitectura distribuida pedida: la adquisición queda separada en la Pico y la ESP32 ejecuta dos rutas de filtrado independientes dentro de su presupuesto de tiempo, con un enlace que delimita bloques, detecta errores y reporta pérdidas. La coincidencia entre el modelo de Python y el firmware muestra que el diseño validado en simulación se trasladó al microcontrolador sin cambiar su comportamiento. Cuando el ruido está separado en frecuencia, el FIR ofrece la mayor reducción con fase lineal y menor latencia, mientras que el método espectral reduce menos pero se adapta por bloque a interferencias cambiantes sin rediseñar coeficientes. Cuando ruido y señal comparten frecuencias, ningún filtro lineal ni ganancia espectral puede separarlos sin quitar información útil, y la elección de parámetros pasa a ser una decisión de compromiso entre ruido residual y distorsión.

VI. RECOMENDACIONES

Se recomienda colocar un filtro antialias analógico antes del ADC de la Pico y medir su respuesta, ya que la protección contra aliasing hoy depende del contenido de la señal. Un DAC externo de mayor resolución o una salida PWM de alta frecuencia mejoraría el rango dinámico de la reproducción, limitado por los 8 bits del DAC interno. En el método espectral convendría estimar el piso de ruido durante silencios y aplicar sustracción espectral con un suelo mínimo de ganancia, para mejorar el caso de ruido superpuesto sin introducir ruido musical. Finalmente, ejecutar cada ruta en un núcleo distinto de la ESP32 permitiría reducir la latencia de extremo a extremo.

REFERENCIAS

[1] Instituto Tecnológico de Costa Rica, "Proyecto grupal 1: Diseño de un sistema distribuido de reducción adaptativa de ruido en tiempo real," CE 1110 Análisis de Señales Mixtas, Cartago, Costa Rica, 2026.

[2] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[3] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2007.

[4] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, 1965.

[5] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, 1978.

[6] J. B. Allen, "Short term spectral analysis, synthesis, and modification by discrete Fourier transform," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 25, no. 3, pp. 235–238, 1977.

[7] Raspberry Pi Ltd, *RP2040 Datasheet: A microcontroller by Raspberry Pi*, Raspberry Pi Ltd, 2024, sec. 4.9, ADC. [Online]. Available: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf.

[8] Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems, 2025, [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

[9] W. W. Peterson and D. T. Brown, "Cyclic codes for error detection," *Proc. IRE*, vol. 49, no. 1, pp. 228–235, 1961.

[10] Espressif Systems, *ESP-IDF Programming Guide v5.5: DAC, SPI Slave Driver and ESP Timer*, Espressif Systems, 2025, [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/.

[11] J. F. Kaiser, "Nonrecursive digital filter design using the I_0-sinh window function," in *Proc. IEEE Int. Symp. Circuits Syst.*, 1974, pp. 20–23.

[12] S. F. Boll, "Suppression of acoustic noise in speech using spectral subtraction," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 27, no. 2, pp. 113–120, 1979.

[13] R. P. Sallen and E. L. Key, "A practical method of designing RC active filters," *IRE Trans. Circuit Theory*, vol. 2, no. 1, pp. 74–85, 1955.

[14] J. Karki, "Analysis of the Sallen-Key architecture," Texas Instruments, Tech. Rep. SLOA024B, 2002.

[15] Texas Instruments, *LM358, LM258, LM2904: Industry-Standard Dual Operational Amplifiers*, Texas Instruments, 2024, sLOS068AB, rev. Oct. 2024.

[16] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[17] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

APÉNDICE A
FIGURAS COMPLEMENTARIAS

Figuras del sistema y del compromiso ruido–distorsión; además, figuras generadas por *prototipo.py* y disponibles en el repositorio:

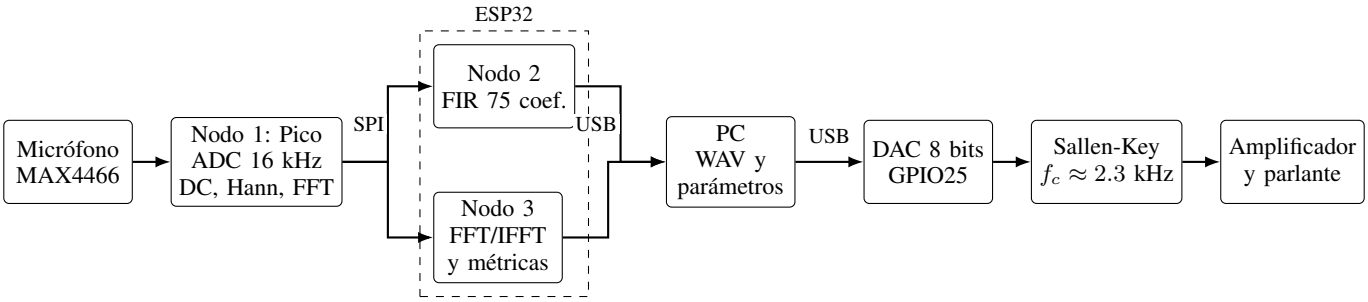


Figura 1. Arquitectura y flujo de datos. Los nodos 2 y 3 comparten la ESP32 con rutas y estado independientes; la PC solo almacena, ajusta parámetros y pide la reproducción.

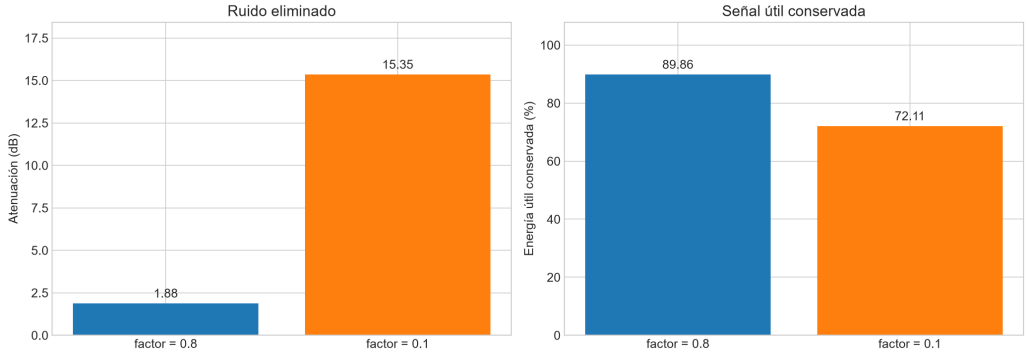


Figura 2. Caso 3 con $f_{min} = 600$ Hz: más supresión implica menos energía útil conservada.

- Diagrama de polos y ceros del FIR Kaiser.
- Respuesta en frecuencia del FIR Kaiser.
- Señales y espectros del caso 2.